Press-A-Button
Voice Typing on Ubuntu

KDE Plasma / Wayland Setup Guide

Using **Handy** + **Input Remapper** + a USB macro button
for hands-free local speech-to-text

Daniel Rosehill
Updated April 2026

Contents

1 Introduction . . . 3
1.1 Why This Guide Exists . . . 3
1.2 Validated Working Configuration (2026-04-14) . . . 3
1.3 Architecture Overview . . . 4
2 Hardware . . . 5
2.1 Single USB Macro Button . . . 5
2.2 USB Foot Pedal . . . 5
2.3 USB Macro Pad . . . 6
3 Installing Handy . . . 8
4 Configuring Handy . . . 9
4.1 General Settings . . . 9
4.2 Choosing a Transcription Model . . . 9
4.3 Advanced Settings . . . 12
5 Installing Input Remapper . . . 15
6 Configuring Input Remapper . . . 16
6.1 Step 1: Select Your Device . . . 16
6.2 Step 2: Create a Preset . . . 16
6.3 Step 3: Map the Button . . . 16
7 Single Button vs Multi-Button Devices . . . 18
7.1 Single Button (Toggle Mode) . . . 18
7.2 Multi-Button Macro Pad . . . 18
8 The Complete Workflow . . . 19
9 Wayland Troubleshooting . . . 20
9.1 Hotkey Doesn’t Fire . . . 20
9.2 Appendix: Why F13 Doesn’t Work (and What Could Fix It) . . . 20
9.2.1 Root Cause . . . 20
9.2.2 What Handy / global-hotkey Could Change . . . 21
9.2.3 Effort / Scope Assessment for a PR . . . 21
10 GPU Acceleration (AMD) . . . 22
10.1 Inference Benchmarks . . . 22
10.2 Viewing Your Own Benchmarks . . . 23
11 System Requirements . . . 24
12 Software References . . . 24

1 Introduction

This guide covers setting up one-press voice typing on Ubuntu 25.10 with KDE Plasma on Wayland using three components:

- **Handy** — a local speech-to-text tool
- **Input Remapper** — a GUI tool for remapping input devices on Linux
- A USB macro button, foot pedal, or macro pad

The end result: press a physical button, speak, and your words appear as typed text wherever your cursor is.

1.1 Why This Guide Exists

Handy works well out of the box on X11, but getting it running smoothly on **KDE Plasma + Wayland** required several non-obvious workarounds:

1. The **overlay had to be disabled** — with it enabled, typed output wouldn't reach the active window.
2. **ydotool had to be manually selected** as the typing tool — the default input method doesn't work on Wayland.
3. **The trigger-key story is messier than you'd expect.** My personal preference — and what I'd still recommend as the ideal — is **F13**: not physically present on almost any keyboard, so emitting it from a USB macro button is guaranteed conflict-free. But as of the last validation date, Handy's shortcut library refuses to register it (`Unknown scancode for key: F13`), and bare single keys like `Pause` on KDE Wayland can silently fail to fire through the XDG GlobalShortcuts portal. The **second-best validated workaround** is `Ctrl+Alt+Space` — a modifier combo tauri accepts and KDE doesn't claim.
4. **Do not switch `keyboard_implementation` to `handy_keys`** (the evdev backend). On this setup it grabs keystrokes and re-injects them through ydotool, causing a runaway loop that floods every focused window with garbage text. Stay on the default `tauri` implementation.

If you're on X11, you may not need all of these steps. On Wayland (the default on modern Ubuntu + KDE), this guide should save you significant troubleshooting.

1.2 Validated Working Configuration (2026-04-14)

The exact setup confirmed working end-to-end — button press → recording → transcription → text at the cursor — on Ubuntu 25.10, KDE Plasma 6, Wayland:

Setting	Value
Handy `transcribe` binding	`ctrl+alt+space`
Handy `keyboard_implementation`	`tauri`
Handy `paste_method`	`direct`
Handy `typing_tool`	`ydotool`
Handy `clipboard_handling`	`copy_to_clipboard`
Handy `overlay_position`	`none`
Handy `push_to_talk`	`false`
Handy `selected_model`	`parakeet-tdt-0.6b-v3`
Input Remapper output	`Control_L + Alt_L + space`
ydotool daemon socket	`/tmp/.ydotool_socket`

Table 1: Validated working configuration

Handy's settings file lives at `~/.local/share/com.pais.handy/settings_store.json`.

1.3 Architecture Overview

Component	Role
USB HID Device	Physical trigger — button, foot pedal, or macro pad
Input Remapper	Maps device button press to `Control_L + Alt_L + space`
Handy	Listens for `Ctrl+Alt+Space`, records audio, transcribes, and types the result via ydotool
ydotool	Wayland-compatible virtual keyboard for typing output into the active window

Table 2: System components and their roles

2 Hardware

Any programmable USB HID device will work. The device just needs to show up as a HID input device on Linux — no special drivers required.

2.1 Single USB Macro Button

The simplest option. A single large button that sends one keycode. Works well with Handy's toggle-to-transcribe shortcut. Available cheaply on Amazon and AliExpress.

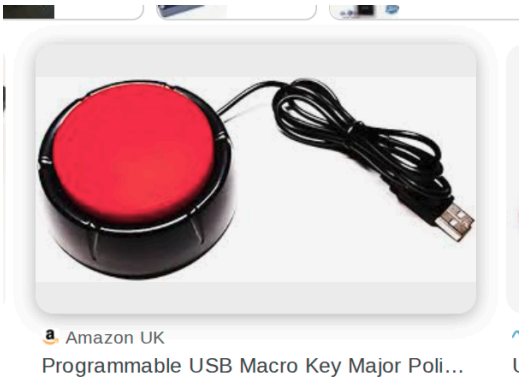

Figure 1: USB macro button available on Amazon

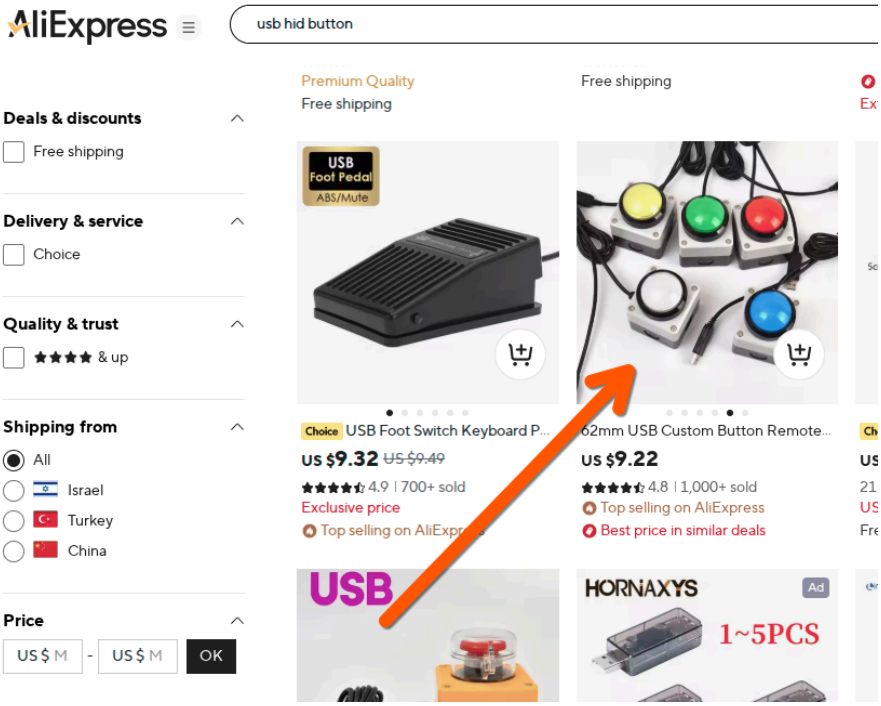

Figure 2: USB macro button listings on AliExpress

2.2 USB Foot Pedal

Hands-free operation with 1–3 keys. Great if you're typing and dictating simultaneously.

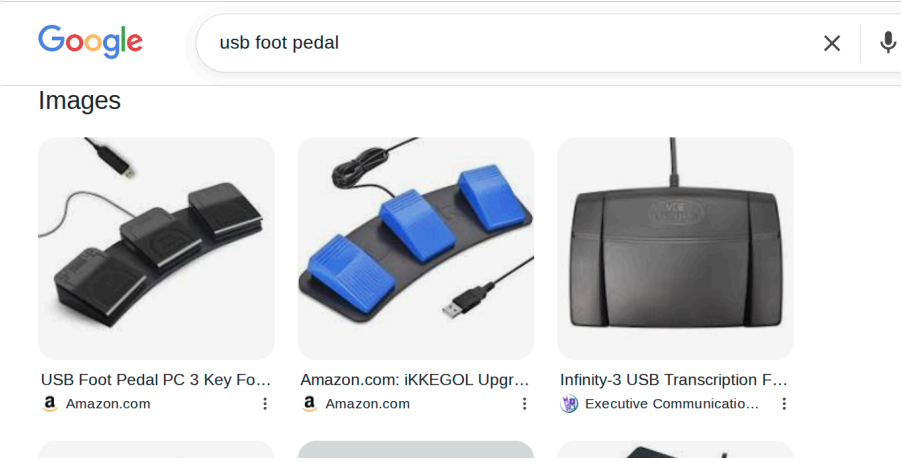

Figure 3: USB foot pedals — various models available

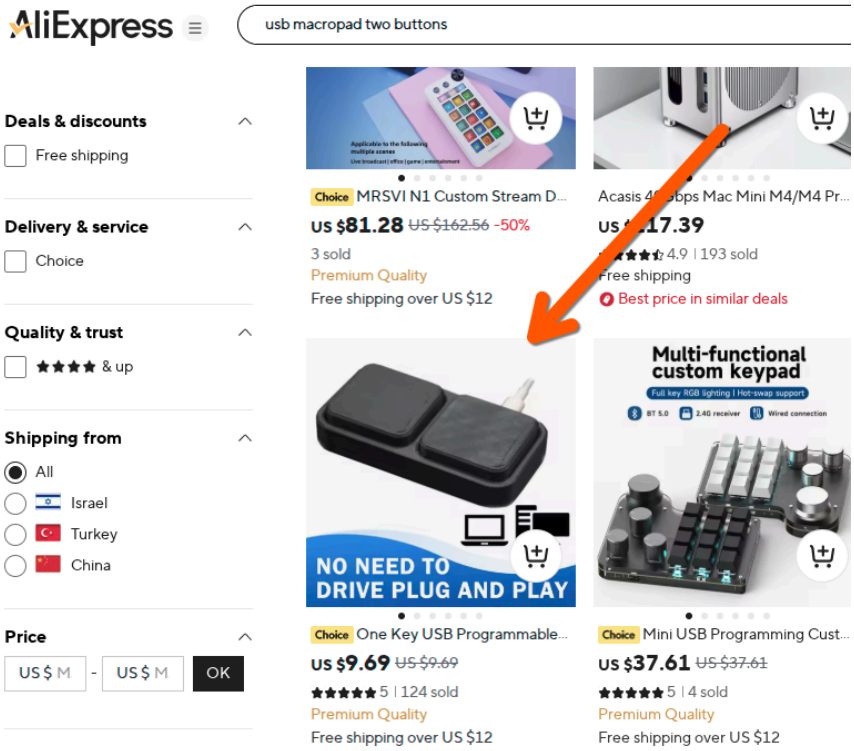

Figure 4: USB foot pedal listings on AliExpress

2.3 USB Macro Pad

The most flexible option. With multiple buttons you can assign separate shortcuts for start, stop, and push-to-talk rather than relying on a single toggle.

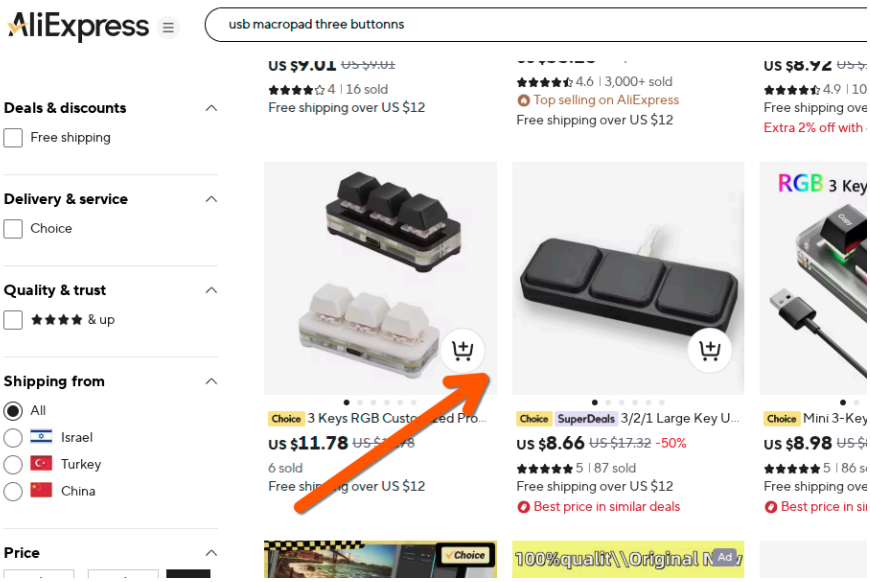

Figure 5: USB macro pads on AliExpress — multi-button options

3 Installing Handy

Install Handy from its GitHub repository or package. It runs as a background app with a system tray icon.

Repository: `https://github.com/cjpais/Handy`

4 Configuring Handy

4.1 General Settings

Configure the following in the **General** tab:

- **Transcribe Shortcut**: Set to `Ctrl+Alt+Space`. Modifier combos register reliably through the XDG portal; bare single keys (F13, Pause, media keys) are hit-and-miss on KDE Wayland.
- **Microphone**: Select your preferred mic
- **Audio Feedback**: Enable for an audible cue when transcription starts/stops

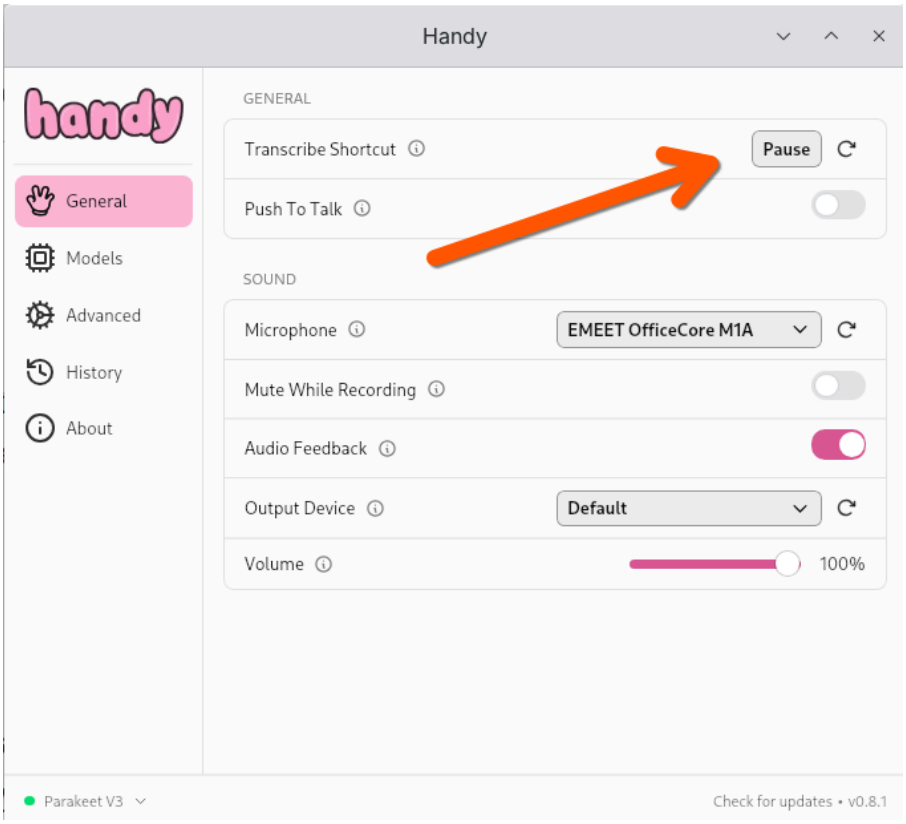

Figure 6: General settings screenshot (from earlier iteration, showing `Pause`). Use `Ctrl+Alt+Space` instead — `Pause` can silently fail to register on KDE Wayland.

4.2 Choosing a Transcription Model

Select a model from the **Models** tab. **Parakeet V3** offers a good balance of accuracy and speed with multi-language support.

Model	Speed	Accuracy	Languages
Parakeet V3 *(recommended)*	Fast	High	Multi-language
Moonshine Base	Very fast	Good	English only
Moonshine V2 Medium	Fast	High	English only
Whisper Small	Fast	Fair	Multi-language
Whisper Medium	Medium	Good	Multi-language
Whisper Large	Slow	High	Multi-language
Whisper Turbo	Fast	Good	Multi-language
SenseVoice	Very fast	Good	CJK + English
Canary 180M Flash	Very fast	Good	Multi-language
Canary 1B v2	Medium	High	25 languages

Table 3: Available transcription models

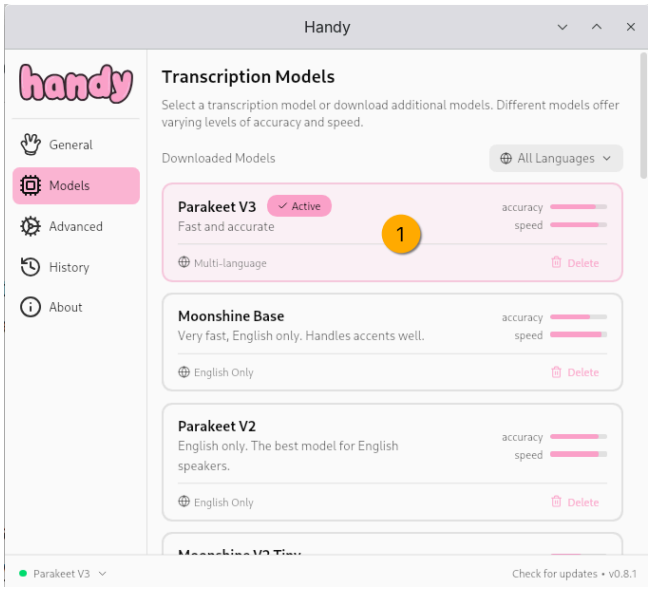

Figure 7: Parakeet V3 selected as the active model

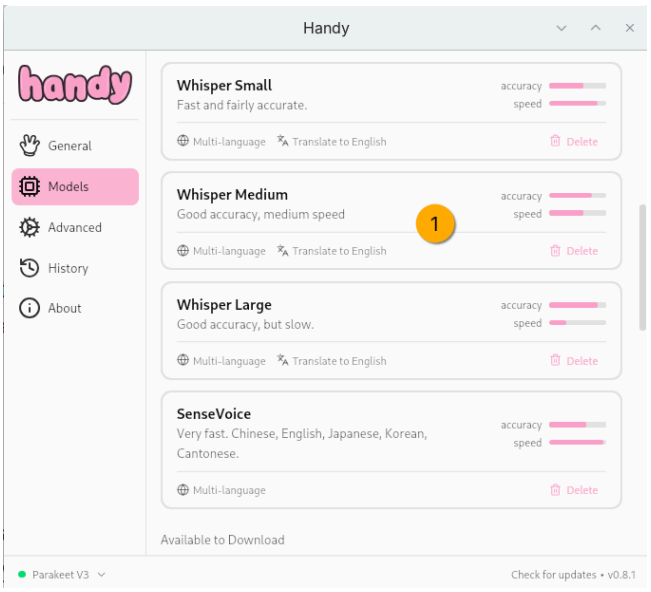

Figure 8: Additional models — Whisper variants and SenseVoice

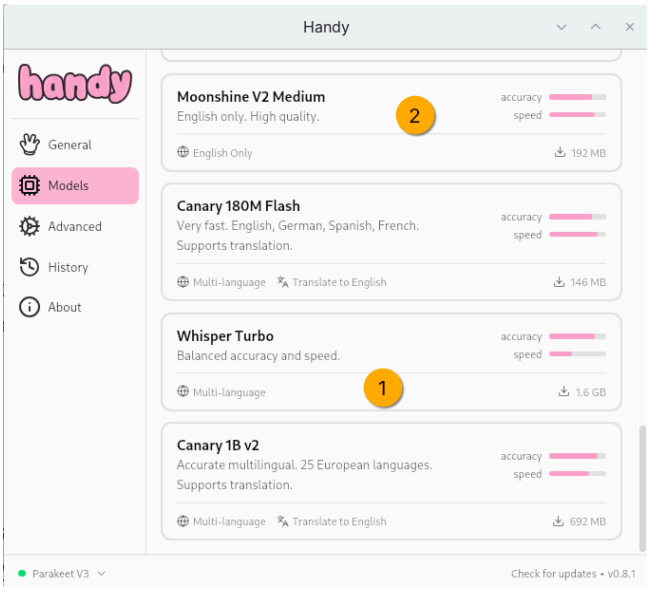

Figure 9: More models — Moonshine V2, Canary, Whisper Turbo

4.3 Advanced Settings

These settings are critical for Wayland compatibility.

Setting	Value	Reason
Start Hidden	On	Runs quietly in the background
Launch on Startup	On	Always available
Show Tray Icon	On	Easy access to settings
Overlay Position	**None**	**Required on Wayland** — overlay prevents typed output reaching the active window
Unload Model	**Never**	Keeps the model in RAM for instant response
Paste Method	**Direct**	Types directly into the active window
Typing Tool	**ydotool**	**Required on Wayland** — the default typing method does not work
Clipboard Handling	Copy to Clipboard	Transcription text also lands on the clipboard as a fallback; switch to “Don’t Modify” if you’d rather preserve the clipboard
History Limit	1 entry	Saves disk space
Auto-Delete Recordings	Keep latest 1	Privacy-friendly

Table 4: Recommended Advanced settings (bold = Wayland-critical)

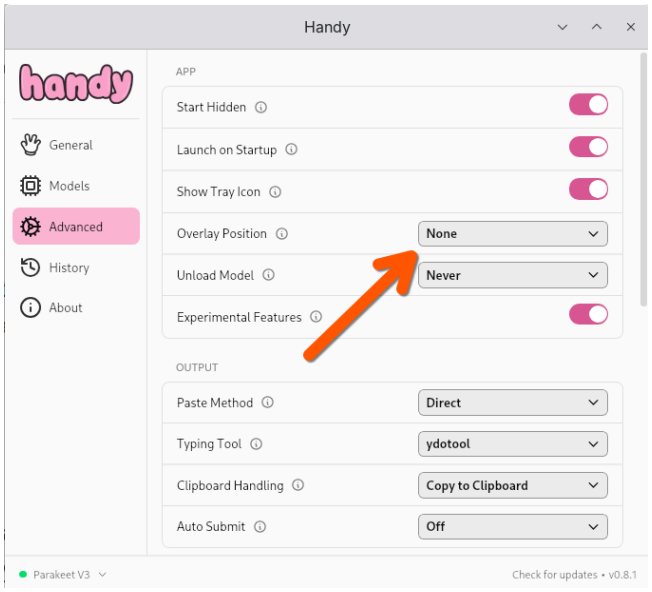

Figure 10: Advanced settings overview — Overlay Position set to None, Typing Tool set to ydotool

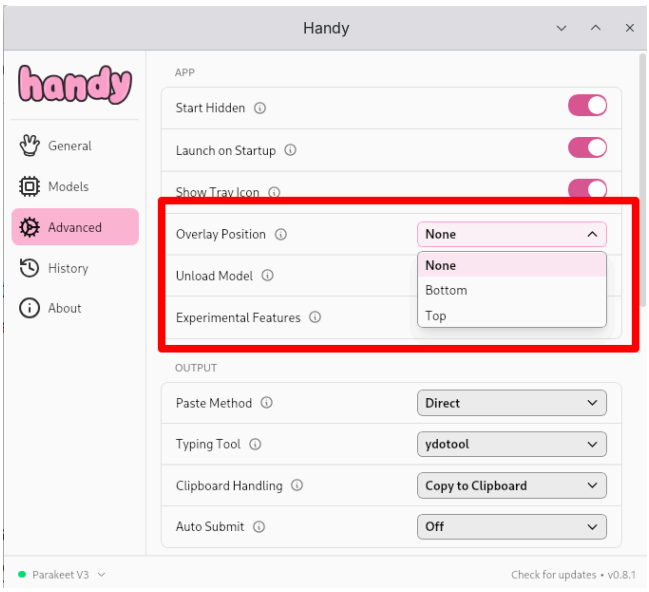

Figure 11: Overlay Position dropdown — select None

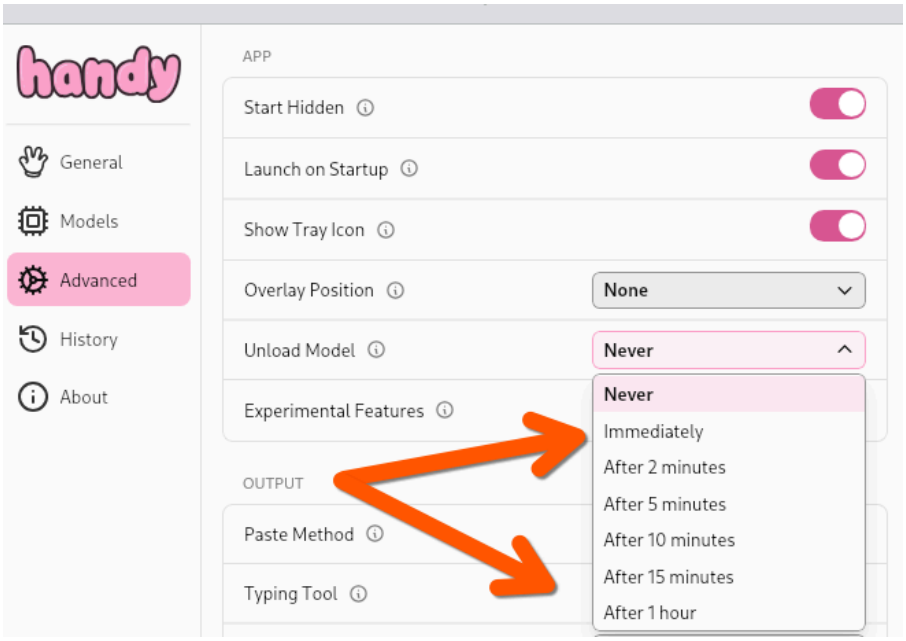

Figure 12: Unload Model dropdown — set to Never

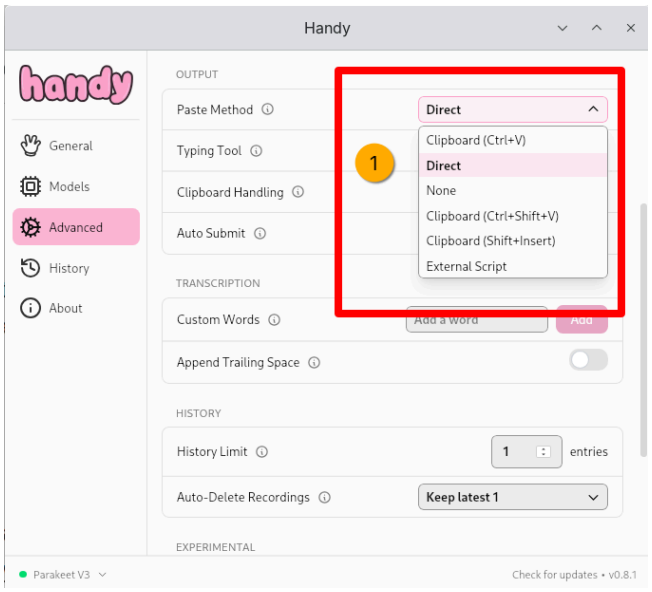

Figure 13: Paste Method dropdown — select Direct

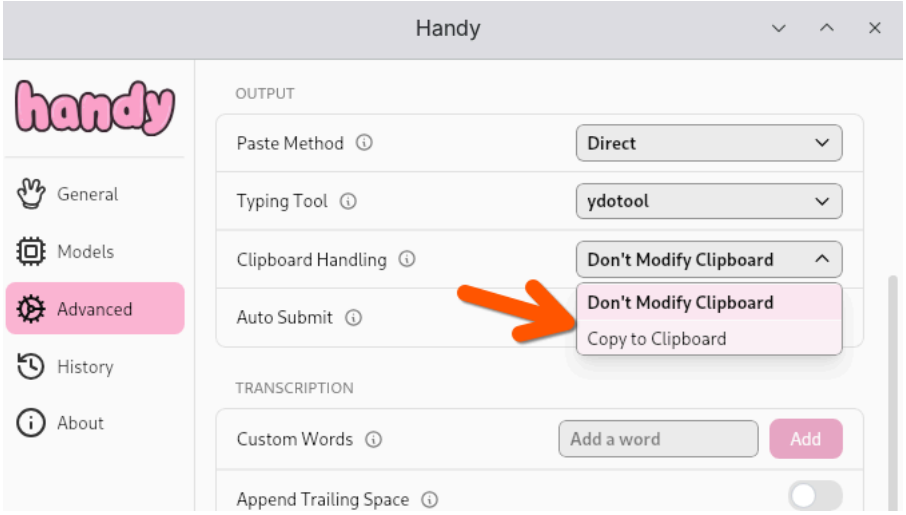

Figure 14: Clipboard Handling — select Don’t Modify Clipboard

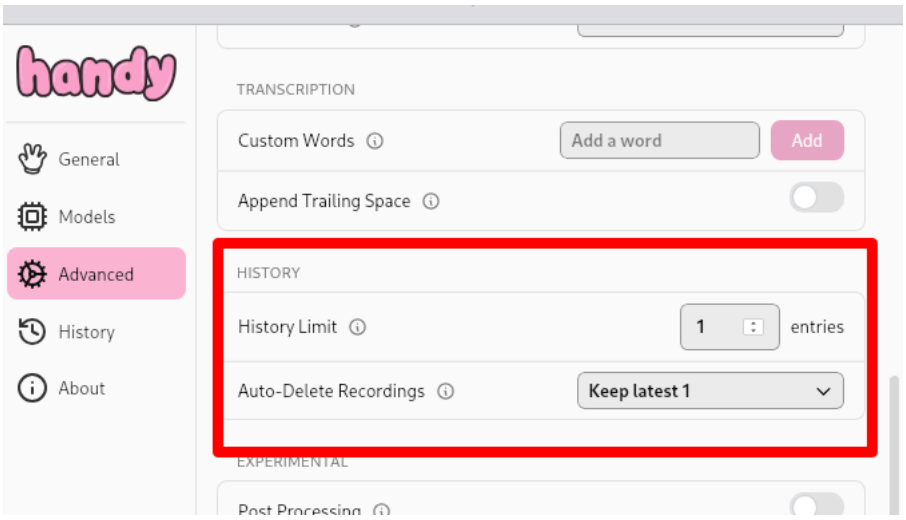

Figure 15: History settings — limit to 1 entry, keep latest 1 recording

5 Installing Input Remapper

Install via apt:

```
sudo apt install input-remapper
```

Input Remapper provides a GUI for remapping any input device on Linux. It runs as a system service and persists mappings across reboots.

6 Configuring Input Remapper

6.1 Step 1: Select Your Device

Open Input Remapper and find your USB device in the **Devices** tab. It will appear by its HID identifier (e.g., `HID 5131:2019`).

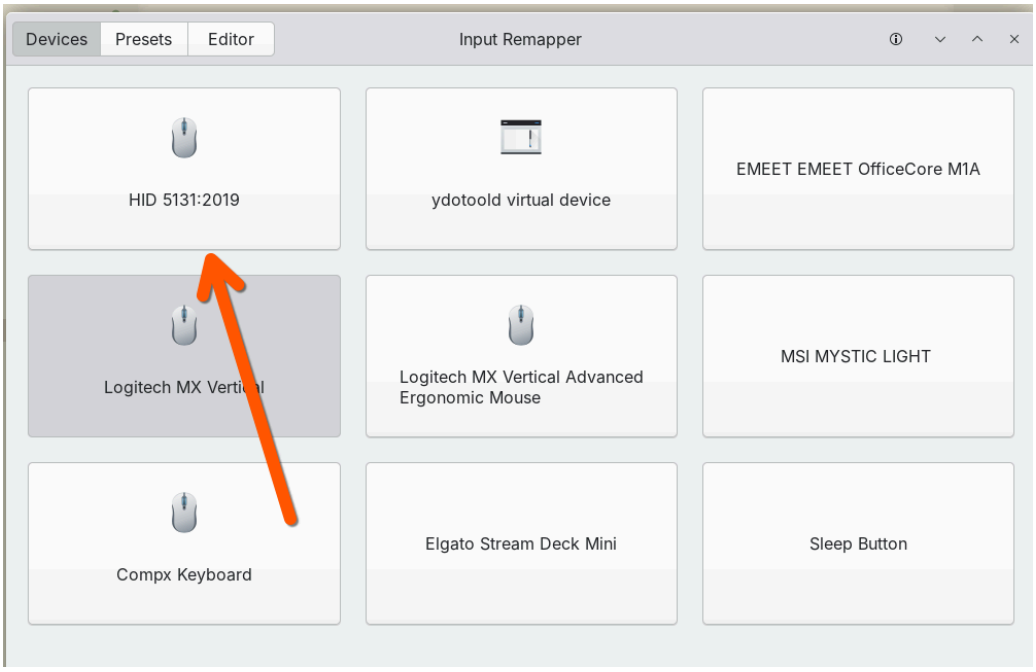

Figure 16: Devices tab — select your USB HID device

6.2 Step 2: Create a Preset

Go to the **Presets** tab and create a new preset (e.g., “USB Voice Typing Trigger”).

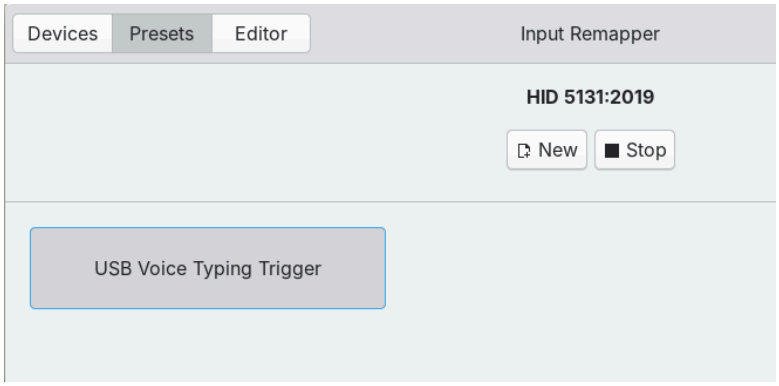

Figure 17: Presets tab — USB Voice Typing Trigger preset created

6.3 Step 3: Map the Button

In the **Editor** tab:

1. Click **Record** and press your USB button to capture the input
2. Set the output type to **Key or Macro**
3. Set the target to **keyboard**
4. In the Output field, enter exactly: `Control_L + Alt_L + space`

5. Enable **Autoload** so the mapping persists across reboots
6. Click **Apply**

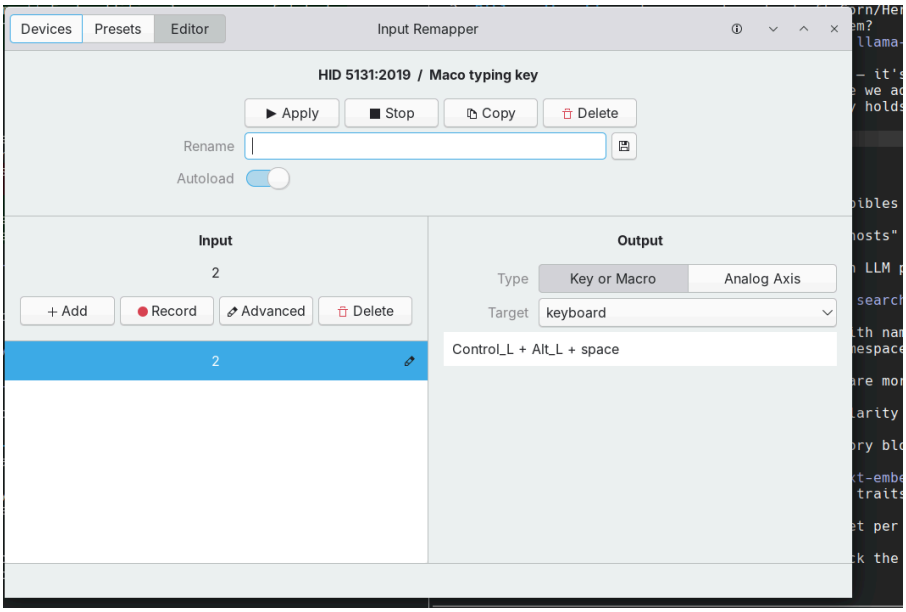

Figure 18: Editor tab — button mapped to `Control_L + Alt_L + space` with Autoload enabled

Why a combo, not a single key? `KEY_F13` would be the ideal single key (absent from almost every physical keyboard, so zero conflicts), but Handy's shortcut library currently rejects it with `Unknown scancode for key: F13`. Single keys like `KEY_PAUSE` can also silently fail to fire on KDE Wayland because the XDG GlobalShortcuts portal doesn't route them. `Ctrl+Alt+Space` goes through cleanly and has been validated end-to-end. See the *Why F13 Doesn't Work* appendix for the technical detail.

7 Single Button vs Multi-Button Devices

7.1 Single Button (Toggle Mode)

With a single button, you use Handy's toggle mode: press once to start transcribing, press again to stop. Simple and effective.

7.2 Multi-Button Macro Pad

With a macro pad (3+ buttons), you can assign separate shortcuts for:

- **Start** transcription
- **Stop** transcription
- **Push-to-talk** (hold to record, release to stop)

This gives finer control and avoids accidentally toggling into the wrong state.

8 The Complete Workflow

Once everything is configured, the workflow is seamless:

1. Handy launches silently at boot and loads the transcription model into memory
2. Input Remapper loads at boot and maps your USB device
3. Place your cursor in any text field
4. Press your button and speak
5. Text appears where your cursor is

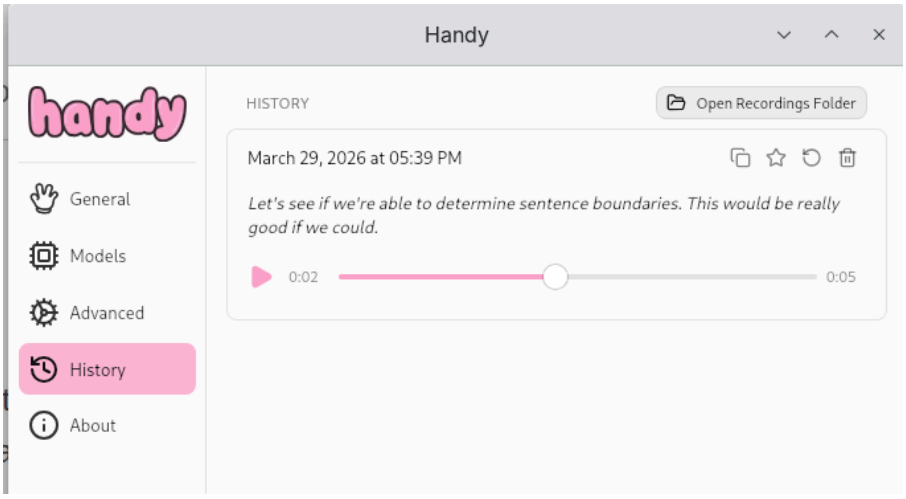

Figure 19: Example transcription result in Handy's History view

9 Wayland Troubleshooting

If transcription runs but no text appears in your application, check the following:

#	Check	Action
1	Overlay disabled	Set Overlay Position to `None` in Advanced settings
2	Typing Tool	Set to `ydotool` in Advanced settings
3	ydotool installed	Run `sudo apt install ydotool` and verify `ydotoold` is active. Default socket is `/tmp/.ydotool_socket` — if you've set `YDOTOOL_SOCKET` in your shell rc files, make sure it points at a socket that actually exists.
4	Paste Method	Set to `Direct` in Advanced settings

Table 5: Wayland troubleshooting checklist

9.1 Hotkey Doesn't Fire

If pressing the hotkey does nothing at all — no Marimba sound, no log entry for the press:

- Check `~/.local/share/com.pais.handy/logs/handy.log` for `register_tauri_shortcut registration error`. `Unknown scancode for key: F13` means Handy's library doesn't know that key — use a modifier combo instead.
- Avoid bare single keys like `Pause`, `ScrollLock`, `Insert`, or media keys on KDE Wayland. They may log as "registered" but never fire because the XDG portal doesn't route them. A `Ctrl+Alt+Space`-style combo is the reliable fix.
- **Do not flip `keyboard_implementation` to `handy_keys`.** On this setup it causes a keystroke injection loop that floods whatever window has focus.
- Sending `SIGUSR1` to the Handy process is *not* a reliable substitute for the hotkey — the signal is received but the handler does not reliably trigger a recording.

9.2 Appendix: Why F13 Doesn't Work (and What Could Fix It)

This section is speculative analysis of Handy's source, not a confirmed diagnosis from the maintainer. It's here to help anyone who'd like to submit a fix upstream.

9.2.1 Root Cause

Handy's default `keyboard_implementation` is `tauri`, which wraps the Rust `global-hotkey` crate. On Linux, `global-hotkey` registers hotkeys via either X11′s `XGrabKey` or the XDG `org.freedesktop.portal.GlobalShortcuts` portal (on Wayland). Both paths translate a named key like `F13` into an X11 keysym or a Linux evdev scancode before registration.

The error surfaced in Handy's log is:

```
Unable to register hotkey: Unknown scancode for key: F13
```

This comes from `global-hotkey`'s Linux keycode-mapping table. Historically that table has covered `F1`–`F12` but omitted `F13`–`F24`, even though the `Code` enum (which mirrors the W3C `KeyboardEvent.code`

spec) *does* define `F13` through `F24`. Result: parsing "F13" into the enum succeeds, but the enum → scancode step returns `None`, and registration fails.

On KDE Wayland specifically, there's a second issue: the XDG GlobalShortcuts portal expects pre-registered actions keyed by an application identifier. Tauri/`global-hotkey` submits a synthetic binding on the fly, which KWin/xdg-desktop-portal-kde may "accept" for modifier combos but silently drop for bare single keys (`Pause`, media keys, etc.) depending on compositor version. That's why `Pause` "registers" in the log but the press never actually reaches Handy.

9.2.2 What Handy / global-hotkey Could Change

Two options, in order of effort:

1. **Extend the Linux scancode table in `global-hotkey` to include `F13–F24`.** This is the minimum fix: add the missing `Code::F13 => 183`, `F14 => 184`, … `F24 => 194` entries (Linux `KEY_F13`–`KEY_F24` evdev constants). One file, a dozen lines. Contributors already mapped F1–F12, so the pattern is established.
2. **Wire Handy's `handy_keys` evdev backend correctly** so it can be used as a safe fallback for compositor-restricted keys. The backend already exists but, on this setup, it triggers a runaway keystroke loop — likely because it reads from `/dev/input/event*` *including* the ydotool-injected virtual keyboard, so every transcription character it types re-triggers the hotkey. Adding a "grab the physical device exclusively" option (`EVIOCGRAB`) or filtering out `uinput`-sourced events would prevent the loop and make `handy_keys` viable for keys the portal won't route.

9.2.3 Effort / Scope Assessment for a PR

Option 1 — Add F13–F24 to `global-hotkey` (upstream):

Repository	`tauri-apps/global-hotkey`
Scope	Linux backend only — `src/platform_impl/linux/...`
Files touched	1–2 (scancode map + a test)
Lines changed	∼20–30
Coding effort	30–60 min
Test effort	1–2 h (X11 + Wayland, extended F-keys are hard to test without extra hardware; input-remapper or `wtype --key F13` can simulate)
Review risk	Low — additive change, existing F1–F12 pattern to follow
Merge effort	Maintainer responsiveness is the main variable; crate is reasonably active

Table 6: Effort assessment — extending scancode map

Once merged upstream, Handy picks it up by bumping its `global-hotkey` dependency in `Cargo.toml` — a one-line change and a fresh build.

Option 2 — Fix `handy_keys` evdev feedback loop (Handy-side):

Repository	`cjpais/Handy`
Scope	`src/shortcut/handy_keys.rs` (or equivalent)
Files touched	1–3
Lines changed	∼50–150
Coding effort	2–4 h
Test effort	2–4 h — need to verify on X11, Wayland, Gnome, KDE, and with various typing tools (`wtype`, `ydotool`, `kwtype`) to ensure no regressions
Review risk	Medium — evdev handling has subtle platform quirks; filtering uinput events requires reading device `ID_INPUT_JOYSTICK`-style properties
Merge effort	Bigger change, more review rounds likely

Table 7: Effort assessment — `handy_keys` loop fix

Recommendation: start with Option 1 upstream — it's a small, well-scoped, additive change that unlocks F13–F24 for every Tauri app, not just Handy. If that lands, the workaround-combo in this guide becomes unnecessary for users who want a clean single-key trigger.

10 GPU Acceleration (AMD)

This setup was tested on an **AMD Radeon RX 7800 XT** (Navi 32, 12 GB VRAM) with ROCm. Handy uses ONNX Runtime for inference and automatically detects the AMD GPU — no manual configuration is needed.

The `ort_accelerator` setting defaults to `auto`, and the log confirms GPU selection:

```
Auto-selected GPU device 0 'AMD Radeon RX 7700 XT (RADV NAVI32)'
(Dedicated, 12288 MB VRAM)
```

10.1 Inference Benchmarks

Benchmarks from Handy's debug log using **Parakeet V3 (INT8)** on the RX 7800 XT:

Recording	Inference	RTF	Transcribed Text
4 sec	574 ms	0.14x	*"Okay, we're mapping out to the pause shortcuts here."*
6 sec	912 ms	0.15x	*"Let's see if we're able to determine sentence boundaries…"*
16 sec	1,695 ms	0.11x	*"It takes a few steps, but being able to enter text seamlessly…"*
4 sec	554 ms	0.14x	*"This text was written."*
24 sec	1,603 ms	0.07x	*"This text was written with parakeet, and the objective…"* (long paragraph)

Table 8: Inference benchmarks — Parakeet V3 (INT8) on AMD RX 7800 XT

Model load time: 1,060–1,870 ms (first load is slower).

A real-time factor (RTF) below 1.0 means inference is faster than real-time. Parakeet V3 on this GPU consistently achieves **0.07–0.15x RTF**, meaning transcription completes in roughly 1/10th the time of the recording.

10.2 Viewing Your Own Benchmarks

Check Handy's log for transcription timing:

```
grep "Transcription completed" \
  ~/.local/share/com.pais.handy/logs/handy.log
```

11 System Requirements

- Ubuntu 25.10 (or similar) with KDE Plasma on Wayland
- A USB HID macro button, macro pad, or foot pedal
- Sufficient RAM for the transcription model (varies by model size)
- **GPU (optional but recommended)**: AMD GPU with ROCm support for accelerated inference. Tested on RX 7800 XT. CPU-only inference also works but will be slower.

12 Software References

Software	Description & Link
Handy	`https://github.com/cjpais/Handy` — Local speech-to-text with direct typing output
Input Remapper	`https://github.com/sezanzeb/input-remapper` — GUI tool for remapping input devices on Linux
ydotool	Wayland-compatible virtual keyboard tool (used by Handy for typing output)

Table 9: Software used in this setup